ECS7012U Music and Audio Programming
A3 Final Project

Adrian Lam - 250018891

April 2026

1 Introduction

This project presents the development and results of Stellaverb, a convolution VST plugin that allows users to generate impulse responses based on ESA Gaia Star Catalogue [4]. Inspired by the original Stellaverb [2] Max MSP patch by Japanese electronic musician and sound designer Katsuhiro Chiba, this implementation explores how astronomical data can be transformed into creative audio effects. By mapping star properties such as distance, direction, and brightness to parameters of an impulse response, the plugin produces unique reverberation textures through convolution.

2 Motivation and Related Work

Stellaverb [2] is originally a Max MSP patch developed by Japanese electronic musician and sound designer Katsuhiro Chiba. The project explores the conceptual idea of simulating the acoustic effect of sound propagating to nearby stars and reflecting back to the listener. This is implemented using the Hipparcos Star Catalogue [3], a catalogue of about 110,000 stars around the solar system observed by the Hipparcos astrometric satellite. This dataset is used to generate a star-field impulse response, where the distance of a star is mapped to the delay time and amplitude attenuation while its direction is mapped to the parameters of 3D panning. Consequently, sounds from more distant stars arrive later and are quieter. Finally the resulting impulse response is then convolved with the input signal to create a star-field reverberation effect. To expand upon this work, I decided to implement this project as a VST plugin with adjustable parameters for impulse response generation, additional signal processing effects, and an interactive graphical interface, making it a flexible sound design tool rather than a conceptual prototype in Max MSP.

3 Methodology

The plugin was developed in four main stages. First, I investigated the star data catalog and tested different subsets of the dataset and mappings to generate impulse responses in Python. Then I implemented a IRGenerator class for generating the impulse response in JUCE and used the JUCE convolution DSP module to load the generated impulse response in the plugin processor. Then I added additional signal processing effects to further shape the resulting wet signal. Finally, I cleaned up the UI to make it look presentable.

3.1 Gathering Star Data and Prototyping in Python

While the original Stellaverb project uses the Hipparcos Star Catalogue, my implementation uses the Gaia Star Catalogue [4] instead. The Gaia Star Catalogue, with the most recent data release in 2022, is much more recent, larger and more precise than the Hipparcos Star Catalogue, which was released in 1997. Gaia provides measurements such as positions, parallaxes, and brightness for nearly 2000 million stars, making it a much bigger dataset than Hipparcos. However, due to its massive size, it is not feasible to use the entire dataset for this plugin. Using the Astropy [1] library, I queried a subset of star data from the Gaia star catalogue, filtering for only the stars with parallax greater than 7 (stars within about 143 parsecs) and parallax_over_error greater than 10 (which is the signal to noise ratio of the parallax, setting it to be greater than 10 ensures reliable distance measurements). The subset ends up being about 900,000 stars, which is much larger than the original Stellaverb but still small enough that it can be processed relatively quickly when generating

```
for (const auto& s : stars)
{
    if (s.distance < minDistance or s.amplitude < minBrightness or s.amplitude > maxBrightness) {
        continue;
    }
    if (s.distance > maxDistance)
    {
        break; // Since the dataset is sorted by distance, we don't have to check any of the stars after
    }

    const float norm = (s.distance - minDistance) / (maxDistance - minDistance);
    int idx = (int)(norm * (irLengthSamples - 1));

    if (idx >= 0 && idx < irLengthSamples)
    {
        float sample = s.amplitude;

        left[idx]  += sample * s.left;
        right[idx] += sample * s.right;
    }

}
```

Figure 1: Main IR generation loop that maps stars to samples.

the impulse response. The exact code used to query and save this subset of data can be found in extras/astropyQueryAndSave.py.

To figure out suitable mappings and parameters, I used Python to quickly generate different impulse responses, then put them into Ableton's convolution reverb to hear what they sound like when convolved with an input signal. Different configurations were tested (see extras/generateImpulse.py), including the original Stellaverb mappings. The final mappings I decided on are similar to Stellaverb, where a star's distance determines its delay time in the IR, its direction being panning, but with its amplitude being controlled by its brightness. In this mapping, the project slightly strays away from being a realistic acoustic simulation of stars but rather a sound design tool driven by star data. Finally, to ensure that the IR generation within the plugin is efficient, I preprocessed some of the star data subset to save on computations in JUCE. This includes pre-calculating the left and right amplitudes of each channel when a star is mapped to stereo based on its direction, converting a star's brightness to amplitude and calculating a star's distance. All of these values are then normalized such that they can be scaled easily in JUCE. The preprocessed dataset is then sorted by distance and packed into a binary file such that it could be read quickly and efficiently with minimal overhead (see extras/astropyPreprocessing.py).

3.2 IR Generation and Convolution in JUCE

Now that we have a usable subset of star data and a desired mapping, we can use it to generate impulse responses in the JUCE plugin. The bulk of the IR generation process is inside the IRGenerator class in IRGenerator.h. The code defines a new struct Star that stores the values of one star unpacked from the binary file created earlier. When a new IRGenerator is constructed, it loads the star data binary file into an array of the struct Star called Stars, which is then used to generate the IR. The IRGenerator class has a few parameters that can be changed to customize how the IR is generated: min and max distance (IR generation only uses stars with distance within the specified distance range), min and max brightness (IR generation only uses stars with distance within the specified brightness range), length (the length which the resulting impulse response will be scaled to), decay (adjusts the tail decay of the IR). Figure 1 shows the main IR generation loop that maps stars to samples. After generating the impulse response, it is then saved to a WAV file inside the application data directory:

```
juce::File::getSpecialLocation(juce::File::userApplicationDataDirectory)
    .getChildFile("Stellaverb")
    .getChildFile("generated_ir.wav");
```

This entire IR generation process is called from the PluginEditor when the user presses the Generate IR button. Despite all the optimizations, generating an impulse response from 900,000 stars still cannot be done every time the user changes a parameter. Therefore, it is only done so when the user sets their desired settings and click the Generate IR button. Once the IR is generated

```
preDelaySmoothed.setTargetValue(preDelay);

for (size_t i = 0; i < numSamples; ++i)
{
    // pre delay (with smoothing to prevent clicks when changing parameter)
    float smoothedPreDelay = preDelaySmoothed.getNextValue();
    float preDelaySamples = smoothedPreDelay * 0.001f * (float) sampleRate;
    preDelayL.setDelay(preDelaySamples);
    preDelayR.setDelay(preDelaySamples);

    float inL = left[i];
    float inR = right[i];

    preDelayL.pushSample(0, inL);
    preDelayR.pushSample(0, inR);

    left[i]  = preDelayL.popSample(0);
    right[i] = preDelayR.popSample(0);
}
```

Figure 2: Code for the reverb pre-delay.

and saved to a WAV file, it calls the prepareToPlay function in the audioProcessor, which loads the impulse response WAV file into the convolution DSP module. To prevent audio glitches during creating and loading the IR, there is a boolean flag called irReady in the audioProcessor that updates based on the status of the IR. When the Generate IR button is clicked, irReady is set to false, which then bypasses all effects and mutes the audio output until the IR is loaded into the JUCE convolution DSP module, then irReady is set back to true, the input signal goes through all effects again, and the audio ouput is unmuted.

3.3 Other Effects

I then implemented several other effects to further shape the sound of the reverb. Unlike IR generation and convolution, these effects will update in real time as the user changes the parameters. In this section, I will describe each of them in order of the signal chain. Before convolving the input signal, I added a pre-delay with smoothing to prevent clicks when changing the parameter (see Figure 2). This is a common parameter found on reverbs that simulates the delay between the direct sound and the reflections arriving to our ears in a real space. Then, I implemented a low pass filter and a high pass filter after convolution to clean up the resulting wet signal. This is done using `juce::dsp::IIR::Coefficients<float>::makeLowPass` and `juce::dsp::IIR::Coefficients<float>::makeHighPass` respectively with Q values 0.707 for a clean Butterworth low pass and high pass. Then saturation is implemented using `juce::dsp::FastMathApproximations::tanh` along side a gain compensation such that it adjusts the signal's tone more than its volume. This version of tanh is chosen as the computation requirements of convolution scale with the length of the impulse response. Previous iterations of the plugin had very high CPU usage when the IR is long and changing `std::tanh` to the approximated version was one of the ways to combat the issue. Next, I implemented stereo widening using simple mid/side widening then mixed the final wet signal with the dry signal, both with gain compensation as well to ensure the volume of the final wet signal is roughly the same as the dry signal (see Figure 3). Finally, an output gain parameter is introduced to adjust the volume of the mixed dry wet signal.

3.4 GUI

The GUI (Figure 4) is divided into three main sections. The top section consists of knobs that control the IR generation parameters and the Generate IR button. These knobs do not update the sound of the reverb until the Generate IR button is clicked, which generates the impulse response and loads it into the JUCE convolution DSP module. The bottom row of knobs controls all other effects that does change the sound of the reverb in real time, including pre-delay, filters and saturation. The waveform of the impulse response is shown in the middle row and acts as a separation for the parameters that update in real time (bottom row) and the ones that do not (top row). To make the plugin look less like a default JUCE plugin, I created a custom look and feel class to change the look of the sliders and the colours of the background and labels.

```cpp
// saturation
float saturationGainComp = 1.0f / std::sqrt(drive);
left[i]  = juce::dsp::FastMathApproximations::tanh(left[i] * drive) * saturationGainComp;
right[i] = juce::dsp::FastMathApproximations::tanh(right[i] * drive) * saturationGainComp;

// stereo widening using mid/side widening
float L = left[i];
float R = right[i];

float mid  = 0.5f * (L + R);
float side = 0.5f * (L - R);

side *= width;

float wideningGainComp = 1.0f / juce::jmax(1.0f, width);

left[i]  = (mid + side) * wideningGainComp;
right[i] = (mid - side) * wideningGainComp;

// dry / wet
const float wetComp = 4.0f; // wet gain compensation since convolution is not level neutral
left[i] = dryL[i] * (1 - dryWet) + left[i] * dryWet * wetComp;
right[i] = dryR[i] * (1 - dryWet) + right[i] * dryWet * wetComp;
```

Figure 3: Code for saturation, stereo widening, and dry/wet mixing.



Figure 4: Final GUI of the Stellaverb VST.

4 Flow Diagram

Figure 5 shows the signal flow diagram of the Stellaverb VST.

5 Results

The plugin is capable of a variety of sounds, from short and long reverbs, to delay and granular-like effects. In this section, we will showcase and analyze a short reverb, a delay/granular-like effect and a long reverb effect using a drum loop and a wurly piano loop. Analysis will focus on IR generation and convolution since that is the core feature of the plugin. Other features will be showcased in detail in the demo video. In each recording, the audio is played three times, the first with the dry input signal, then the second time with the wet output signal only, then the final time with the dry wet parameter at 0.5 to blend both signals.

5.1 Drums - Short Reverb

Let us first examine the generated IR (Figure 6) using Sonic Visualiser to see if it was generated correctly based on the parameters set by the user. The parameters used were IR length of 0.41 seconds, 5.87 decay time, and no restrictions on the stars, meaning all stars are used to generate the IR. We see in Figure 6 that the length of the IR is 0.41 seconds as expected, the tail decays quickly, and the impulses are densely spaced since all stars are used in the IR generation. In the recording spectrogram (Figure 7), we see that the wet signal (middle) looks like the dry signal (left) but slightly smeared, which is what we expect when convolving a dry signal with a short and dense IR with decay.

5.2 Wurly Piano - Delay / Granular

Again, we will examine the generated IR (Figure 8) using Sonic Visualiser to see if it was generated correctly based on the parameters set by the user. The parameters used were IR length of 5 seconds, 0.95 decay time, and utilizing only stars with distance within the range 0.14 to 0.16 and brightness within the range 0.16 to 0.18. Since the distance range and brightness range is small, there are only several impulses present in the IR, which is what we expect. Looking at the spectrogram (Figure 9), we see that the wet signal (middle) looks like the dry signal (left) but with a delay-like effect, which is what happens when convolving a dry signal with a sparse IR. In the bottom row parameters, the low pass cutoff is set to 5853 Hz during the recording. We can use this information to further confirm that the plugin is convolving the signal correctly. In the dry wet mixed signal (right), we clearly see a tail that rolls off at around 6000 Hz and lasts 5 seconds.

5.3 Wurly Piano - Long Reverb

The IR parameters used for this recording are the same as the wurly piano delay above, but with no restrictions on the star data. We see that the generated IR (Figure 10) has the same length of 5 seconds, but with densely spaced impulses. In the spectrogram of the audio recording (Figure 11), we see in the wet signal (middle) that the convolution heavily smears the dry signal (left), which is what we would expect as the IR used is long and densely spaced. Again, we see the 5 second tail that rolls off around 6000 Hz in the dry wet signal (right).

6 Limitations and Future Work

While I am happy with how Stellaverb turned out, there are some limitations to be resolved that could not be completed due to time constraints. Currently, the plugin only works in stereo as the processBlock code assumes there are two channels. Another limitation is the high CPU usage as mentioned above. However, this is a result of utilizing convolution since it is a very computationally demanding process, especially with a long impulse response. One limitation is that because the IR file loaded into the convolution module is hardcoded to the same file every time, if a DAW session has multiple instances of the plugin with different IRs loaded, and the user closes and reopens the session, all instances of the plugin will load with the most recent generated IR. One solution could be to use different file names for each instance of the plugin, or to directly pass in the impulse response AudioBuffer into the JUCE convolution DSP module instead of saving and loading a WAV

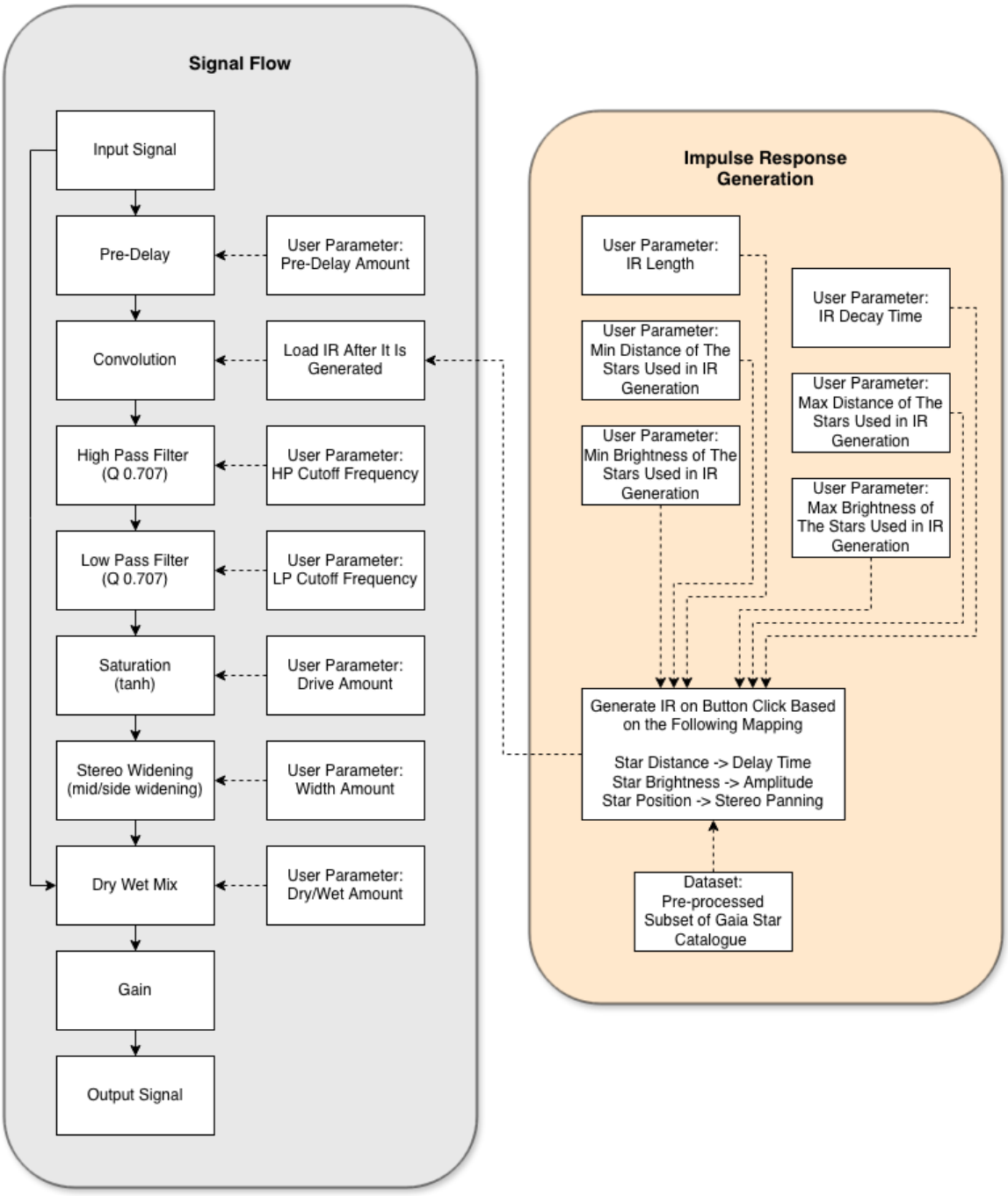


Figure 5: Signal flow diagram of Stellaverb VST.

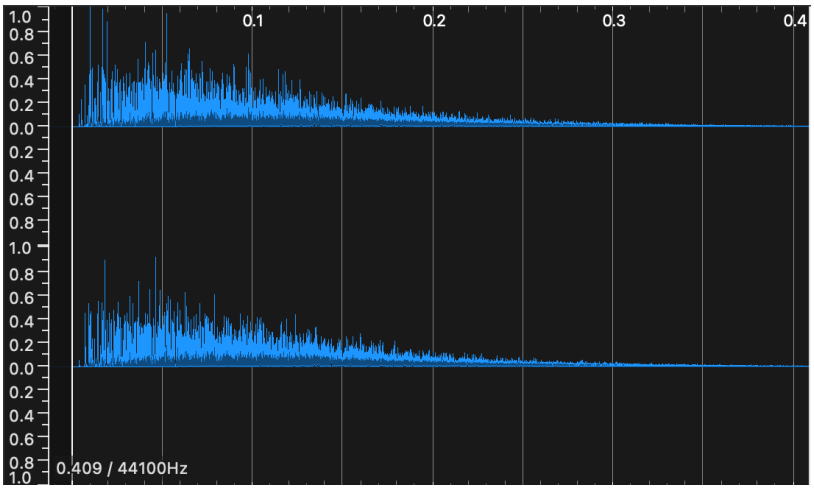


Figure 6: Plot of the generated short reverb IR. The x axis of the plot is time and the y axis is amplitude. The top half is the left channel and the bottom half is right channel.

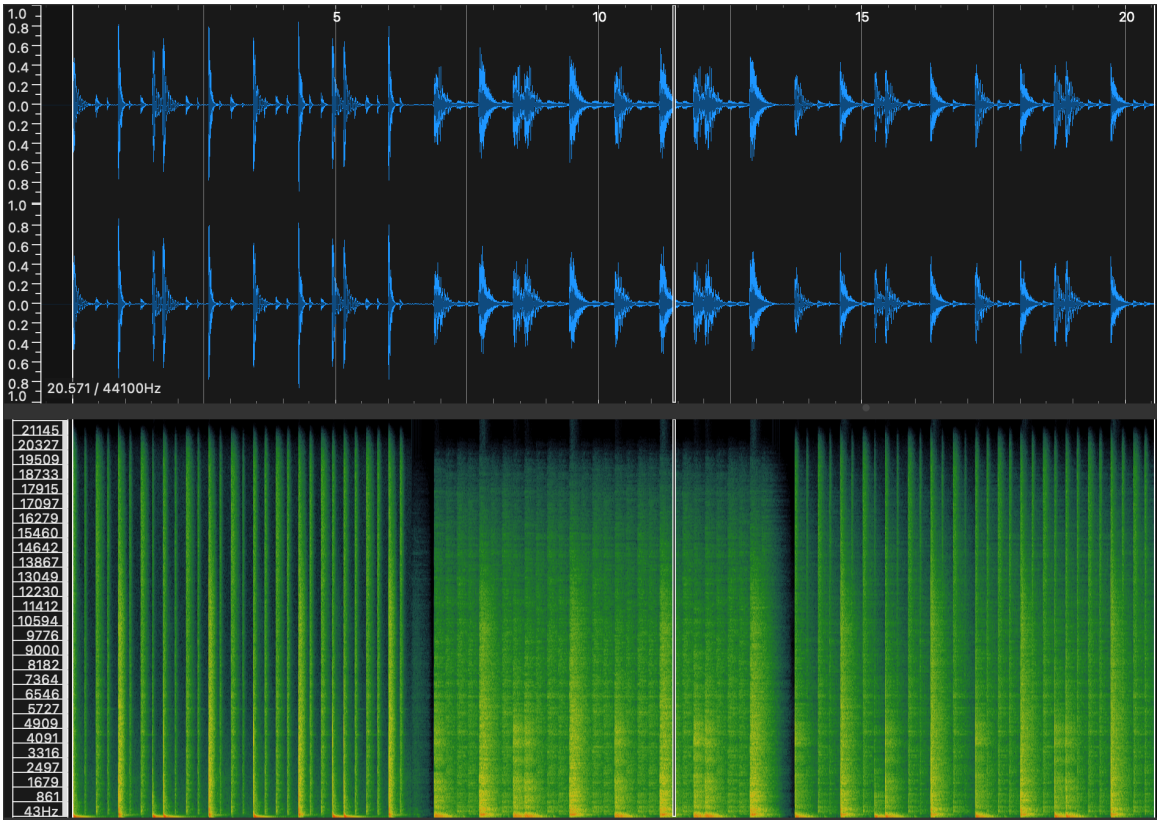


Figure 7: Waveform (top) and spectrogram (bottom) of the drums short reverb recording. In both the waveform and spectrogram, the x axis represents time. In the waveform, the y axis represents amplitude while it represents frequency in the spectrogram.

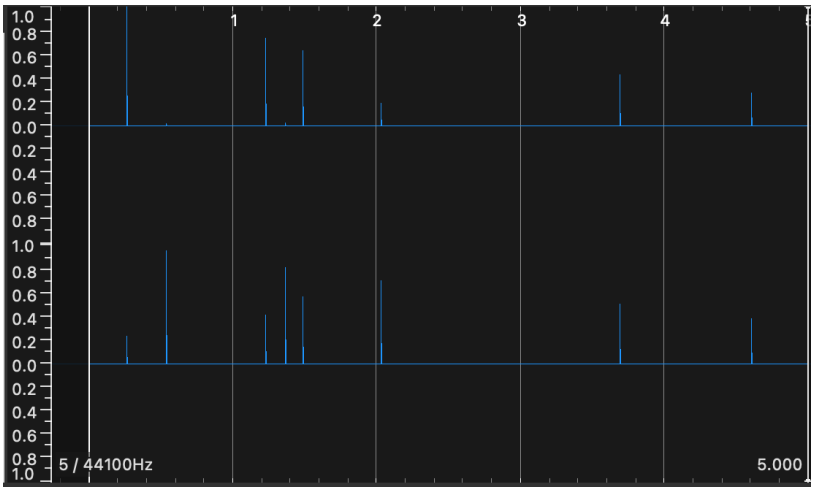


Figure 8: Plot of generated delay-like IR. The x axis of the plot is time and the y axis is amplitude. The top half is the left channel and the bottom half is right channel.

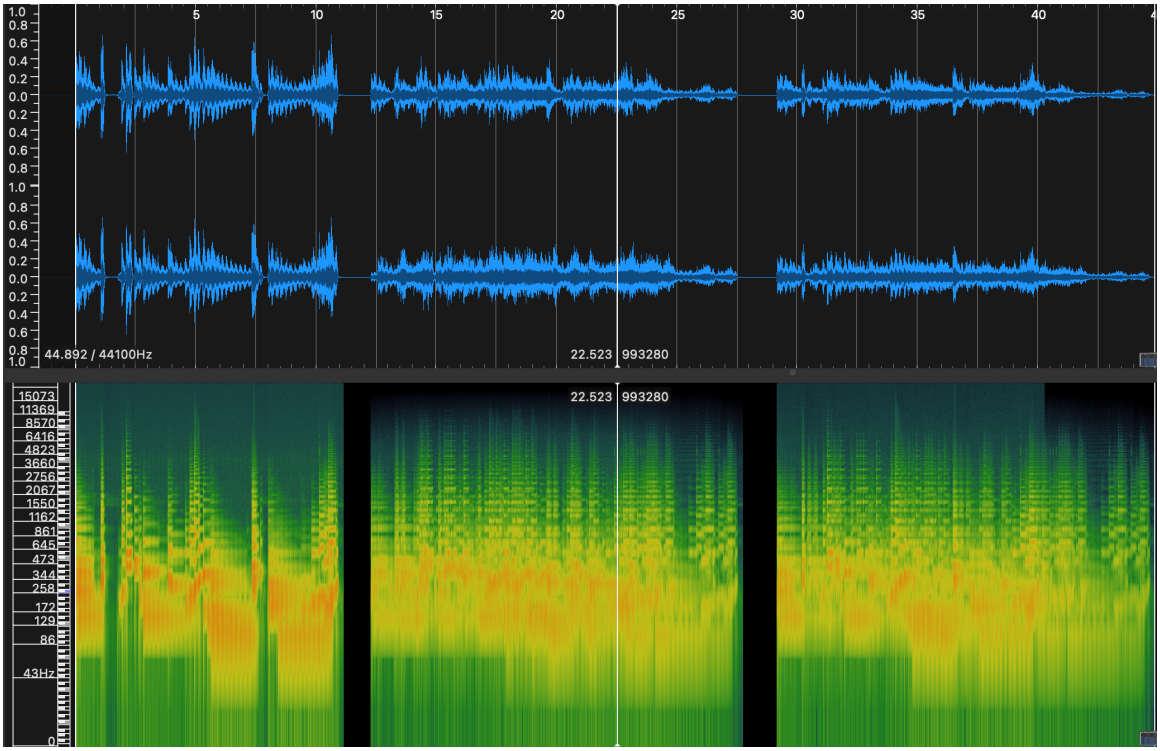


Figure 9: Waveform (top) and spectrogram (bottom) of the wurly piano delay recording. In both the waveform and spectrogram, the x axis represents time. In the waveform, the y axis represents amplitude while it represents frequency in the spectrogram.

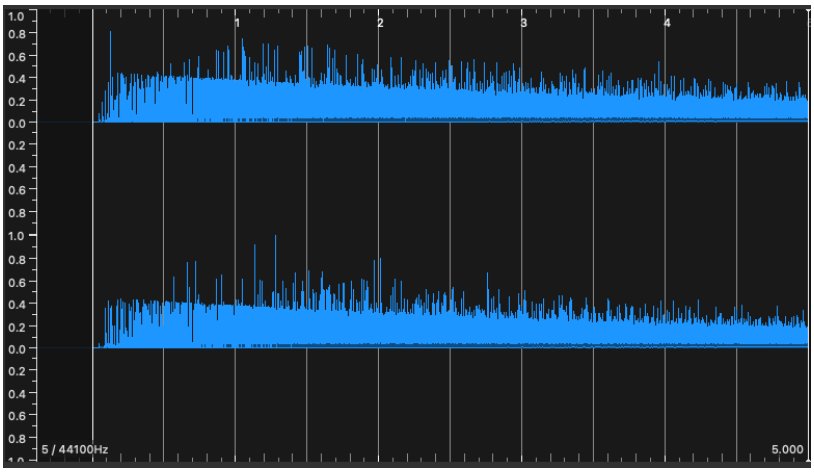


Figure 10: Plot of generated long reverb IR. The x axis of the plot is time and the y axis is amplitude. The top half is the left channel and the bottom half is right channel.

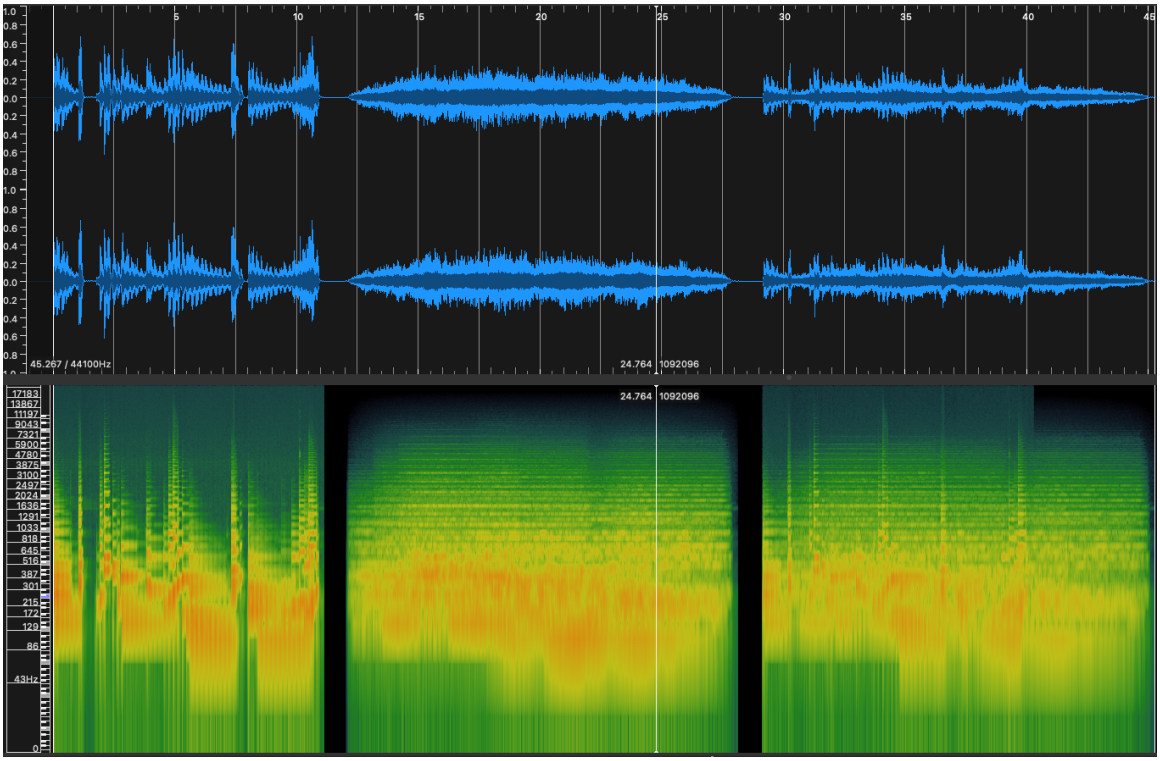


Figure 11: Waveform (top) and spectrogram (bottom) of the wurly piano long reverb recording. In both the waveform and spectrogram, the x axis represents time. In the waveform, the y axis represents amplitude while it represents frequency in the spectrogram.

file first. There are also a few UI bugs that would like to mention in this report. One is that the waveform display does not scale to the full length of the IR when it is generated until the user closes and reopens the GUI, then the waveform display scales itself to fit the full length. Another minor issue is that currently, the user is allowed to generate an IR with the max distance being less than the min distance, resulting in an empty IR. The same also applies to min brightness and max brightness. This could be fixed by using JUCE's two valued slider, allowing the user to pick a valid range, which would also be more intuitive to the user compared to the current sliders. In terms of future work, some features I would like to implement are visualization of the star data in the UI for improved user experience and understanding of how the stars are being usedin IR generation, 3D panning which was present in the original Stellaverb Max MSP patch, a preset manager and a way to save the generated IR to a different file for future use.

7 Conclusion

In this project, a functional and extensible implementation of Stellaverb was developed as a VST plugin using JUCE. By leveraging a subset of the Gaia star catalogue [4], the system successfully generates impulse responses that produce a wide range of reverberation effects, from dense ambient spaces to sparse delay-like textures. The results demonstrate that mapping non-audio datasets to audio processing parameters can yield both technically valid and creatively useful outcomes.

References

[1] Astropy Collaboration, T. P. Robitaille, E. J. Tollerud, et al. Astropy: A community python package for astronomy. *Astronomy & Astrophysics*, 558:A33, 2013.

[2] Katsuhiro Chiba. Stellaverb, July 2020. Accessed: 2026-04-13.

[3] ESA. *The Hipparcos and Tycho Catalogues*. ESA SP-1200. ESA Publications Division, Noordwijk, The Netherlands, 1997.

[4] Gaia Collaboration, A. Vallenari, A. G. A. Brown, et al. Gaia data release 3. *Astronomy & Astrophysics*, 667:A1, 2022.